

ADR 0132 — A number is claimed by a push that can fail, not by a read that cannot

- **Status:** Accepted — implemented, mutation-proved two ways failing differently
- **Date:** 2026-09-06
- **Issue:** #697 (follow-up to #689)
- **Extends:** [ADR 0130](#), whose own Alternatives section named this exact fix and deferred it: *"Worth reconsidering if collisions keep recurring after this ADR's two mitigations ship — the table above would then be evidence that 'make lateness cheap' was not enough."*
- **Supersedes / superseded by:** — (complements 0130; the heading census and `scripts/adr-renumber.sh` stay the recovery path for whichever lane skips reservation and collides anyway)

Context

ADR 0130 shipped two mitigations for the same-day collision problem (0121/0125/0126/0127/0128, five in one day) and named a third it declined: reserving a number atomically before starting work, rejected because "it requires every lane's workflow to change ... and a reservation-conflict story of its own (two lanes racing to reserve is the same race one level up, just faster)."

0130 shipped, and the same day, the collision recurred twice more — on *its own PR*:

1. #87's bisect ADR read `docs/adr/README.md` on a branch 103 commits behind main, computed 0121, and lost that number to #591's animation ADR sometime in the intervening window. Renumbered to 0129, then lost *that* number to #584's settings-surface ADR, working the same window. Renumbered to 0130.

2. While #87's branch was mid-renumber, #694 — the PR carrying 0130's own `scripts/adr-renumber.sh` and heading census — merged, and it *also* claimed 0130. #87's now-provisional 0130 needed a third renumber, to 0131, using the very tool that PR had just introduced.

Six collisions in one day now, one of them on the fix for the first five. That is the recurrence 0130's own Alternatives section set as the bar for revisiting reservation.

0130's stated objection to reservation — "the same race one level up, just faster" — is correct that a naive reservation scheme (a lane reads a shared reservations file, decides a number is free, writes its own row back) has exactly the same read-then-write gap the numbering process itself has, only over a smaller file. It is not an argument against reservation; it is a requirement on how reservation must be built: **the claim itself must be a single atomic operation, with no read-decide-write gap of its own.**

Decision

The claim is a `git push`, not a read. `docs/adr/RESERVED.md` is a new, append-only ledger. `scripts/adr-reserve.sh <branch> [title]`:

1. Fetches `origin/main`.
2. Computes the next number as one past the higher of `docs/adr/README.md`'s highest real entry and `docs/adr/RESERVED.md`'s highest pending one — both, not just the first (decision 2 below is why this matters).
3. Builds a single commit, in a throwaway detached worktree off `origin/main`, appending one row naming the number, a UTC timestamp, the claiming branch, and a working title.
4. Pushes that commit directly to `origin/main` with `git push origin HEAD:main`.

Step 4 is the whole mechanism. `git push` to a ref that has moved since the pusher's last fetch is refused outright — a non-fast-forward rejection, not a merge, not a "last write wins." If two lanes race, at most one push can ever land against the pre-race state; the other is rejected by the remote before it can be applied, with no window in which both believe they succeeded. This is the same guarantee every write this app's own planner relies on for a shared repository (ADR 0019); it costs nothing extra to get here, because git already enforces it on every push, whether or not this script exists.

1. Losing the race is not an error — it is a retry signal

A rejected push means exactly one thing: `origin/main` moved. The script re-fetches and recomputes from scratch, rather than trying to resolve a conflict in the file — a file that is pure append has nothing to merge, and the number the loser had guessed may now be wrong for reasons having nothing to do with the winner (an unrelated `main` advance moves the goalposts identically). Retrying from a fresh read is simpler and more correct than any attempt to patch a stale guess in place, and it is bounded (`MAX_ATTEMPTS=10`) so a genuinely wedged repository fails loud rather than looping forever.

2. The next number is the max of two tables, not one

A number can be "spoken for" two ways: it has a real ADR merged (`docs/adr/README.md`), or a lane has reserved it and not yet merged (`docs/adr/RESERVED.md`). Computing the next number from `README.md` alone reintroduces a race that git's push atomicity does *not* protect against by itself: two lanes, reserving sequentially rather than concurrently (lane A reserves 0132 and pushes; lane B starts later, fetches the now-current `main`, and computes "next"), would both compute 0132 again if `RESERVED.md`'s own pending rows are invisible to that computation — and lane B's push would **succeed**, because appending a second `| 0132 | ... |` row is not a conflicting edit to the same line, it is two independent, non-overlapping appends. Git's non-fast-forward check protects the *file*, never the *meaning* of what

two authors wrote into it. This is exactly the failure the mutation proof's second arm demonstrates.

3. A reservation is a promise with a UTC timestamp, not a permanent claim

`scripts/adr-release.sh <number>` removes an abandoned reservation's row, using the same atomic-push mechanism, and refuses outright if `docs/adr/README.md` already has a real entry for that number — releasing a number that landed for real would be actively destructive, not merely useless. The normal cleanup path does not go through this script at all: the PR that lands the real `docs/adr/NNNN-slug.md` deletes its own `RESERVED.md` row in the same commit, the number's home becoming the real table. Release exists only for the case where that PR never happens.

4. This is prevention; #689/#694's tools remain the recovery

Nothing here removes `adr_index_matches_the_files.rs`'s heading census or `scripts/adr-renumber.sh`. A lane that skips reservation — or whose reservation predates this ADR's own adoption, as #87's did — can still collide, and the fix is still a one-command renumber, not a five-place hand-edit. Reservation and renumbering are independent: one prevents, the other recovers, and neither needs the other to work.

On dogfooding this ADR's own number

This document's own number (0132) was **not** claimed with `scripts/adr-reserve.sh`, deliberately not attempted: `docs/adr/RESERVED.md` does not exist on `origin/main` until this PR merges, and the script requires it to already be there. Claiming this ADR's own number is circular on day one in exactly the way #697's own dispatch anticipated. 0132 was picked the old way — reading `docs/adr/README.md` and `docs/adr/RESERVED.md` on current `main` immediately before writing this document, same as every ADR before this one — and avoiding the two numbers already known to be pending elsewhere (0130, landed for real under

#694 while this branch was open; 0131, claimed by #87's own in-flight renumber). The first real use of `scripts/adr-reserve.sh` for an ADR of its own will be whichever lane claims 0133 or later.

Alternatives considered

A lock service or shared mutex (a file lock, a database row, an external coordination service). Rejected: git's own push semantics already provide exactly the atomicity needed — a second mechanism that duplicates it would be strictly more moving parts for no additional guarantee, and would need its own failure story (what happens when the lock holder crashes?) that a stateless retry loop does not.

Poll `RESERVED.md` for a free slot instead of computing "next." Rejected: this issue's numbering is sequential by convention (ADR 0031), not slot-based — there is no fixed set of numbers to poll for freedom in, only an ever-climbing maximum. Polling would also reintroduce a read-then-act gap of its own between "observed free" and "claimed."

Skip the ledger file; encode the reservation in the branch name or a GitHub API call (a draft PR, an issue label).

Rejected: this needs to work for any lane, including ones with only local git push access and no GitHub API token configured (per this session's own credential-scoping work, ADR 0122/0126/0129) — a plain `git push` is the one operation every lane already has.

Require reservation before ANY ADR work, enforced by CI.

Rejected as out of scope for #697: this ADR makes reservation available and documents it as the new first step, but does not add a CI gate refusing an ADR whose branch never reserved. #87's own bisect ADR is the concrete case for why not — its number was picked before this mechanism existed, and retrofitting a hard gate would have made that legitimate, already- in-flight work newly non-compliant. A CI enforcement gate is a reasonable future addition once every active lane's tooling has adopted the new step, not a same-PR requirement.

Consequences

- Every lane handoff prompt now names `scripts/adr-reserve.sh` `<branch> [title]` as the first step before starting ADR work, printing the claimed number to use — not "check the README, take the next free number."
(`docs/handoffs/README.md` and `docs/adr/README.md`'s own intro both updated in this PR.)
- `docs/adr/RESERVED.md` is a new, append-only, git-tracked file. A landed ADR's own PR is responsible for deleting its row in the same commit; an abandoned one is cleaned up with `scripts/adr-release.sh`, which refuses to touch a number that turned out to be real.
- The next-number computation now reads two files, not one — a lane reading only `docs/adr/README.md` by habit (old muscle memory, or a script that predates this ADR) will under-count and risk exactly the collision this ADR closes; `scripts/adr-reserve.sh` is the one place that computation should happen from here on.
- ADR 0130's heading census and `scripts/adr-renumber.sh` are unchanged and remain the recovery path — nothing here assumes every lane adopts reservation immediately.
- This ADR's own number (0132) was claimed the old way, for the documented circularity reason above — a known, one-time exception, not a precedent for skipping reservation going forward.

Mutation proof

`failure-atlas`'s `mutation_check` targets compiled Rust reached by `cargo test`; this mechanism is a bash script orchestrating real `git push` calls against a repository, which is outside what that tool drives. Checked first per the standing exception, then proved by hand against `scripts/test-adr-reserve.sh` — a real integration test that builds a throwaway local bare repository (never real GitHub) and forces a genuine, deterministic non-fast-forward rejection using a `git` wrapper on `PATH` that delays one lane's

`push` call, rather than hoping two background processes happen to collide by scheduling luck (an earlier, undelayed version of this same test never once observed a real rejection across dozens of runs — both lanes' full fetch-build-push pipelines routinely completed faster than a second subshell could even start). Both mutations applied to `scripts/adr-reserve.sh`, reverted after, `diff` confirmed byte-identical to the pre-mutation file both times:

arm	mutation	mutated result
no retry	loop bound " <code>\$MAX_ATTEMPTS</code> " (10) replaced with a literal <code>1</code>	caught, reliably across 3 runs: the losing lane's push is rejected once and the script gives up — <code>test-adr-reserve.sh</code> 's concurrent-race assertion fails because the loser produces no number at all (<code>rc=unknown</code> , empty stdout), a crash rather than a duplicate
RESERVED.md ignored	<code>next=\$((10#\$readme_max > 10#\$reserved_max ? ...))</code> replaced with <code>next=\$((10#\$readme_max + 1))</code>	caught, reliably across 3 runs, by a different assertion: the sequential (non-racing) claim test — reserve once, then reserve again for a different branch after the first landed — gets the SAME number twice; both pushes report success (<code>rc=0</code> for both), since two independent appends of an identical-looking row are not a conflicting edit git's own atomicity has any reason to reject

The two arms fail at disjoint assertions for a reason that matters: the first arm breaks the atomicity of a single claim (git's own

guarantee, lost when nothing retries past a real rejection); the second breaks the uniqueness of claims *across time*, a property git's push semantics were never going to provide by themselves — no amount of retrying protects against a next-number computation that cannot see reservations git has already durably accepted. Proving both mattered: a fix for one leaves the other's failure mode completely open.

The test itself needed two real bugs found and fixed before it could prove anything, both worth naming since a green integration test with either bug still in it would have reported "distinct numbers" while actually observing a crashed lane or a purely sequential, non-concurrent run:

- The original race used `(...) & ; echo "$?" >rc & — a ;` inside a single line with a trailing `&` backgrounds only the last command, so the first lane's entire pipeline ran to completion, synchronously, before the second lane's subshell was ever started. There was no concurrency in what looked like a concurrency test. Fixed by grouping the subshell and its exit-code capture inside one parenthesised block before backgrounding it.
- The delay wrapper matched `[ "$1" = "push" ]`, but the script's actual push invocation is `git -C <dir> push ... — $1` is `-C`, never `push`. The delay never fired; both mutations above were initially invisible because the "race" never manifested a rejection to retry from. Fixed by searching the full argument list for the subcommand rather than assuming its position.

Both were caught by cross-checking a fixed, non-mutated run against direct inspection of each side's `stdout / stderr / exit code` rather than trusting a passing summary line — the "green test that proves nothing" failure mode this project has hit before, here in test infrastructure being built rather than in the invariant it was meant to prove.